

UNIVERSIDAD DEL VALLE DE GUATEMALA

Ingeniera de Software II - Sección 30

Lynette García



Plataforma digital para la colaboración interdisciplinaria entre asociaciones estudiantiles universitarias

Angel Gabriel Sanabria Morales - 24725

Saul Esteban Castillo Arenas - 24915

Vernel Josué Hernández Cáceres - 24584

Derek Friedhelm Coronado Chilin - 24732

Samuel Antonio Robledo López - 241282

Guatemala, 2026

Product Backlog

La siguiente tabla presenta la pila del producto completa de UVG Collab. Para cada historia se indica su prioridad, su estimación en puntos de historia, el sprint en que fue terminada y su estado al cierre del Sprint 5. Las historias marcadas como «Semestre 2 (backlog)» permanecen en la pila sin asignación de sprint.

Resumen: 69 ítems · 258 puntos de historia totales · 40 historias completadas (158 SP, 61 % del producto) · 29 pendientes (100 SP).

Historias Técnicas (HT)

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HT-01	Preparación Sprint 1	Alta	5	Sprint 1	Completada
HT-02	Tareas transversales Sprint 2	Alta	5	Sprint 2	Completada
HT-03	Tareas transversales Sprint 3	Alta	5	Sprint 3	Completada
HT-04	Tareas transversales Sprint 4	Alta	3	Sprint 4	Completada
HT-05	Tareas transversales Sprint 5	Alta	3	Sprint 5	Completada

Infraestructura y Configuración

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-00	Configuración e infraestructura del proyecto	Alta	13	Sprint 1	Completada
HU-62	Hosteo en servidor Azure con deploy automatizado	Alta	5	Sprint 3	Completada

Autenticación y Perfil

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-01	Registro e inicio de sesión	Alta	8	Sprint 1	Completada
HU-02	Completar perfil académico	Alta	5	Sprint 2	Completada
HU-13	Editar perfil académico	Alta	2	Sprint 2	Completada
HU-14	Recuperación de contraseña	Media	3	Sprint 3 / 5	Completada
HU-15	Subir y actualizar foto de perfil	Baja	2	Sprint 4	Completada
HU-16	Cerrar sesión de forma segura	Alta	1	Sprint 1	Completada

Exploración y Descubrimiento de Proyectos

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-03	Explorar proyectos publicados	Alta	5	Sprint 1	Completada
HU-04	Ver detalle de proyecto	Alta	5	Sprint 1	Completada
HU-17	Búsqueda avanzada de proyectos	Alta	5	Sprint 2	Completada
HU-18	Proyectos recomendados según perfil	Media	5	Semestre 2 (backlog)	Pendiente

HU-19	Filtrar proyectos por organización	Media	3	Sprint 4	Completada
HU-59	Ver proyectos destacados	Media	3	Sprint 3 / 5	Completada

Postulaciones

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-05	Postularse a un rol en proyecto	Alta	5	Sprint 1	Completada
HU-06	Consultar mis postulaciones	Alta	5	Sprint 1	Completada
HU-20	Cancelar postulación enviada	Media	2	Sprint 3 / 5	Completada
HU-21	Notificación de resultado de postulación	Media	3	Sprint 2	Completada
HU-22	Notificación al líder de nueva postulación	Media	2	Sprint 3 / 5	Completada
HU-23	Retiro de colaborador del proyecto por el líder	Baja	2	Semestre 2 (backlog)	Pendiente
HU-24	Salir voluntariamente de un proyecto	Baja	2	Semestre 2 (backlog)	Pendiente

Gestión de Proyectos (Líder)

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-07	Crear y publicar proyecto	Alta	8	Sprint 2	Completada
HU-08	Gestionar postulaciones recibidas	Alta	5	Sprint 2	Completada
HU-25	Cambiar estado del proyecto y ver Mis Proyectos	Media	3	Sprint 2	Completada
HU-26	Editar proyecto existente	Media	3	Sprint 3	Completada
HU-27	Eliminar o archivar proyecto	Baja	2	Sprint 4	Completada
HU-28	Ver equipo actual del proyecto	Alta	3	Sprint 3	Completada

Gestión de Tareas y Seguimiento

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-09	Vista general de tareas del proyecto	Media	3	Sprint 5	Completada
HU-29	Crear y asignar tareas a colaboradores	Media	5	Sprint 5	Completada
HU-30	Visualizar tablero	Media	5	Sprint 5	Completada

	Kanban de tareas				
HU-31	Comentar en tareas	Baja	2	Sprint 5	Completada
HU-32	Seguimiento de avance general del proyecto	Media	3	Sprint 5	Completada
HU-67	Etiquetas y prioridades visibles en las tarjetas del tablero	Media	2	Sprint 5	Completada

Evidencias y Validación

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-10	Subir evidencias de trabajo	Baja	3	Semestre 2 (backlog)	Pendiente
HU-11	Validar horas de participación	Baja	3	Semestre 2 (backlog)	Pendiente
HU-33	Revisar y aprobar evidencias	Baja	2	Semestre 2 (backlog)	Pendiente
HU-34	Generar certificado de participación	Baja	3	Semestre 2 (backlog)	Pendiente
HU-35	Exportar historial de participación	Baja	2	Semestre 2 (backlog)	Pendiente

Organizaciones Estudiantiles

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-36	Crear organización estudiantil	Media	5	Semestre 2 (backlog)	Pendiente
HU-37	Gestionar miembros de organización	Media	3	Semestre 2 (backlog)	Pendiente
HU-38	Ver perfil público de organización	Media	3	Semestre 2 (backlog)	Pendiente

Comunicación y Notificaciones

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-12	Notificaciones generales del sistema	Media	3	Sprint 3	Completada
HU-39	Configurar preferencias de notificación	Baja	2	Semestre 2 (backlog)	Pendiente
HU-40	Mensajería interna entre usuarios	Baja	8	Semestre 2 (backlog)	Pendiente

Moderación y Administración

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-41	Panel de administración de usuarios	Media	5	Sprint 4	Completada
HU-42	Moderar proyectos y contenido	Media	3	Semestre 2 (backlog)	Pendiente

HU-43	Gestionar catálogos del sistema	Media	3	Semestre 2 (backlog)	Pendiente
-------	---------------------------------	-------	---	----------------------	-----------

Analítica y Reportes

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-44	Ver estadísticas generales de uso	Baja	5	Semestre 2 (backlog)	Pendiente
HU-45	Reportes de actividad por organización	Baja	5	Semestre 2 (backlog)	Pendiente
HU-46	Historial de cambios de estado de proyectos	Baja	2	Semestre 2 (backlog)	Pendiente

Gamificación

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-47	Sistema de puntos e insignias por participación	Baja	5	Semestre 2 (backlog)	Pendiente
HU-48	Ranking de estudiantes más activos	Baja	3	Semestre 2 (backlog)	Pendiente
HU-49	Logros desbloqueables por hitos	Baja	3	Semestre 2 (backlog)	Pendiente

Portafolio y Reconocimiento

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-50	Portafolio público del estudiante	Baja	5	Semestre 2 (backlog)	Pendiente
HU-51	Historial de proyectos completados	Baja	3	Sprint 4	Completada
HU-52	Valoraciones entre colaboradores	Baja	3	Semestre 2 (backlog)	Pendiente

Calendario y Planificación

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-53	Calendario de actividades del proyecto	Baja	5	Semestre 2 (backlog)	Pendiente
HU-54	Recordatorios de fechas límite	Baja	2	Semestre 2 (backlog)	Pendiente

Configuración y Experiencia de Usuario

ID	Historia de usuario	Prioridad	SP	Sprint de cierre	Estado
HU-55	Tema visual (modo claro/oscuro)	Media	3	Sprint 4	Completada
HU-56	Diseño responsivo para dispositivos móviles	Media	5	Semestre 2 (backlog)	Pendiente

HU-57	Tutorial o guía de uso interactiva	Media	3	Semestre 2 (backlog)	Pendiente
HU-58	Seguir creadores de proyectos	Baja	2	Semestre 2 (backlog)	Pendiente
HU-60	Feedback en actividades del proyecto	Baja	3	Semestre 2 (backlog)	Pendiente
HU-61	Autocompletar correo con carné o apellido	Media	2	Sprint 3 / 5	Completada

Nota sobre historias trabajadas en dos sprints

Las historias HU-14, HU-20, HU-22, HU-59 y HU-61 aparecen con sprint de cierre «Sprint 3 / 5». Estas historias se dieron por terminadas en el Sprint 3, pero la validación posterior en el entorno productivo evidenció que no eran funcionales de extremo a extremo: HU-14 dependía de un proveedor de correo inoperante sobre el dominio institucional, y HU-20, HU-22, HU-59 y HU-61 presentaban defectos de autenticación, de identidad del actor notificado y de integración entre frontend y backend. En el Sprint 5 se retomaron y se cerraron con validación en producción. Se documentan explícitamente en ambos sprints para mantener la trazabilidad y no inflar la velocidad reportada.

Como medida correctiva, a partir del Sprint 6 la Definición de Terminado del equipo se amplía: una historia solo pasa a estado Done cuando ha sido verificada en el entorno de producción, no únicamente en local.

Sprint Backlog

ID / Clave Jira	Tipo	Título y descripción	S P	Horas	Responsable	Inicio	Fin plan.	Cierre real
HT-05 <i>IESUC-223</i>	Historia	Tareas transversales Sprint 5 (fix producción, pruebas y entrega) <i>Resolver los bugs de producción conocidos y preparar la entrega del sprint para que el incremento sea demostrable en el ambiente desplegado.</i>	3	—	Angel Sanabria	06/07	28/07	28/07
T-89 <i>IESUC-235</i>	Subtarea	Corregir NEXT_PUBLIC_API_URL en deploy.yml y validar build de producción <i>Corregir la variable en blanco en el workflow, redeployar vía GitHub Actions y validar que el frontend consuma la API a través de Nginx en la VM de Azure.</i>	—	3	Angel Sanabria	06/07	08/07	08/07
T-90 <i>IESUC-236</i>	Subtarea	Pruebas de integración y preparación de entrega Sprint 5 <i>Ejecutar pruebas de integración del módulo de tareas completo (crear, asignar, mover, comentar, ver avance) en local y en producción, y preparar los artefactos de entrega.</i>	—	3	Samuel Robledo	25/07	27/07	20/07
HU-29 <i>IESUC-224</i>	Historia	Crear, asignar y editar tareas del proyecto <i>Como líder de proyecto, quiero crear, asignar y editar tareas con prioridad y fecha límite, para organizar el trabajo del equipo dentro de la plataforma.</i>	5	—	Vernel Hernández	06/07	14/07	18/07
T-91 <i>IESUC-237</i>	Subtarea	Backend: módulo tasks con CRUD de tareas del proyecto <i>Modelo Prisma (título, descripción, estado, prioridad, fecha límite, asignado, proyecto), migración y endpoints</i>	—	6	Vernel Hernández	06/07	10/07	15/07

		<i>CRUD sobre /proyectos/:id/tareas con DTOs de validación y guard de membresía.</i>						
T-92 <i>IESUC-238</i>	Subtarea	Frontend: modal de creación y edición de tareas <i>Modal en Next.js con título, descripción, selector de asignado limitado a miembros del proyecto, prioridad y fecha límite. Validaciones en cliente, estados de carga y toasts.</i>	—	4	Vernel Hernández	11/07	14/07	15/07
HU-30 <i>IESUC-225</i>	Historia	Tablero Kanban del proyecto con drag & drop <i>Como miembro de un proyecto, quiero visualizar las tareas en un tablero Kanban y moverlas entre columnas arrastrándolas, para gestionar el flujo de trabajo de forma visual.</i>	5	—	Samuel Robledo	11/07	19/07	20/07
T-93 <i>IESUC-239</i>	Subtarea	Backend: endpoint de cambio de estado y orden de tareas <i>Endpoint PATCH /tareas/:id/estado con los estados del tablero y campo de orden para persistir la posición dentro de la columna. Validar transiciones y membresía del proyecto.</i>	—	4	Samuel Robledo	11/07	14/07	20/07
T-94 <i>IESUC-240</i>	Subtarea	Frontend: tablero Kanban con drag & drop (dnd-kit) <i>Columnas por estado con @dnd-kit, tarjetas con avatar del asignado, prioridad y fecha límite con semáforo. Actualización optimista al soltar y rollback si el PATCH falla.</i>	—	6	Samuel Robledo	15/07	19/07	20/07
HU-67 <i>IESUC-226</i>	Historia	Etiquetas y prioridades visibles en las tarjetas del tablero <i>Como miembro de un proyecto, quiero asignar etiquetas de color y prioridad a las tareas, para identificar visualmente el</i>	2	—	Vernel Hernández	15/07	18/07	26/07

		<i>tipo y la urgencia del trabajo en el tablero.</i>						
T-95 <i>IESUC-241</i>	Subtarea	Modelo Etiqueta y chips de color y prioridad en tarjetas <i>Modelo Prisma Etiqueta (nombre, color, proyecto) con relación N:M a tareas, endpoints CRUD, y render de chips de color e ícono de prioridad en las tarjetas del tablero.</i>	—	4	Vernel Hernández	15/07	18/07	26/07
HU-09 <i>IESUC-227</i>	Historia	Vista «Mis tareas» transversal a todos mis proyectos <i>Como estudiante colaborador, quiero ver en un solo lugar todas las tareas que tengo asignadas en cualquier proyecto, para priorizar mi trabajo sin entrar proyecto por proyecto.</i>	3	—	Saul Castillo	08/07	13/07	14/07
T-96 <i>IESUC-242</i>	Subtarea	Backend: endpoint GET /usuarios/me/tareas <i>Devuelve todas las tareas asignadas al usuario autenticado en cualquier proyecto, con filtros por estado y ordenamiento por fecha límite, incluyendo datos del proyecto en la respuesta.</i>	—	3	Saul Castillo	08/07	10/07	13/07
T-97 <i>IESUC-243</i>	Subtarea	Frontend: vista «Mis tareas» agrupada por proyecto <i>Vista /dashboard/mis-tareas que consume el endpoint anterior, agrupa por proyecto con contadores e indicador visual de vencimiento (vencida, próxima, a tiempo).</i>	—	3	Saul Castillo	11/07	13/07	13/07
HU-31 <i>IESUC-228</i>	Historia	Comentarios en tareas del proyecto <i>Como miembro de un proyecto, quiero comentar en las tareas, para discutir detalles y dejar contexto sin salir de la plataforma.</i>	2	—	Derek Coronado	09/07	12/07	10/07

T-98 <i>IESUC-244</i>	Subtarea	Backend y UI de comentarios en tareas <i>Endpoints POST, GET y DELETE sobre /tareas/:id/comentarios con autor y timestamp, e hilo de comentarios en el detalle de la tarea con avatar, hora relativa y contador en la tarjeta.</i>	—	4	Derek Coronado	09/07	12/07	10/07
HU-32 <i>IESUC-229</i>	Historia	Barra de avance del proyecto según tareas completadas <i>Como líder de proyecto, quiero ver el porcentaje de avance del proyecto calculado a partir de sus tareas, para monitorear el progreso de un vistazo.</i>	3	—	Derek Coronado	15/07	20/07	11/07
T-99 <i>IESUC-245</i>	Subtarea	Backend: cálculo de % de avance y barra de progreso <i>Cálculo de tareas hechas sobre total, expuesto en GET /proyectos/:id y en el listado, con desglose por estado y barra de progreso en cabecera y tarjetas. Caso de cero tareas sin errores.</i>	—	6	Derek Coronado	15/07	20/07	28/07
HU-14 <i>IESUC-230</i>	Historia	Recuperación de contraseña mediante notificación al administrador <i>Como usuario registrado, quiero recuperar el acceso a mi cuenta enviando mi carné y correo institucional, de modo que un administrador reciba la solicitud dentro de la plataforma y me haga llegar un enlace de restablecimiento.</i>	3	—	Angel Sanabria	09/07	13/07	15/07
T-100 <i>IESUC-246</i>	Subtarea	Registrar solicitud de recuperación y notificar a administradores <i>Persistir la solicitud en el modelo SolicitudRecuperacion, notificar a los administradores con la plantilla correspondiente y</i>	—	6	Angel Sanabria	09/07	13/07	15/07

		<i>conservar la validación del token JWT de tipo reset en el endpoint de restablecimiento.</i>						
HU-20 <i>IESUC-231</i>	Historia	Cancelar postulación a un proyecto <i>Como estudiante postulante, quiero cancelar una postulación que envié, para retirarme de un proyecto antes de ser aceptado.</i>	2	—	Samuel Robledo	06/07	09/07	09/07
T-101 <i>IESUC-247</i>	Subtarea	Endpoint DELETE /postulaciones/:id y botón de cancelar <i>Permitir cancelar solo si la postulación sigue en estado pendiente y pertenece al usuario autenticado. Botón con modal de confirmación en «Mis postulaciones» y toast de confirmación.</i>	—	4	Samuel Robledo	06/07	09/07	09/07
HU-22 <i>IESUC-232</i>	Historia	Notificación al líder cuando llega una nueva postulación <i>Como líder de proyecto, quiero recibir una notificación cuando alguien se postula a mi proyecto, para revisarla y responder rápido.</i>	2	—	Derek Coronado	21/07	24/07	10/07
T-102 <i>IESUC-248</i>	Subtarea	Notificación automática al líder al recibir postulación <i>Al crear una postulación, generar notificación para el líder que enlace a la vista de postulaciones y emitirla en tiempo real vía Socket.io si está conectado. No se notifica al propio postulante.</i>	—	4	Derek Coronado	21/07	24/07	28/07
HU-61 <i>IESUC-233</i>	Historia	Autocompletar correo institucional en el registro <i>Como estudiante que se registra, quiero que el sistema autocomplete mi correo institucional a partir de mi carné, para registrarme más rápido y sin errores de tipeo.</i>	2	—	Saul Castillo	14/07	17/07	14/07

T-103 <i>IESUC-249</i>	Subtarea	Autogenerar correo institucional en el formulario de registro <i>Sugerir el correo @uvg.edu.gt a partir del carné o apellido, dejando el campo editable, y validar que solo se acepten correos del dominio institucional al enviar.</i>	—	4	Saul Castillo	14/07	17/07	11/07
HU-59 <i>IESUC-234</i>	Historia	Proyectos destacados en el dashboard <i>Como estudiante, quiero ver una sección de proyectos destacados en el dashboard, para descubrir oportunidades de colaboración relevantes sin buscar manualmente.</i>	3	—	Angel Sanabria	20/07	24/07	17/07
T-104 <i>IESUC-250</i>	Subtarea	Sección «Destacados» en el dashboard <i>Endpoint GET /proyectos/destacados y sección con tarjetas de proyecto enriquecidas en el dashboard, con un máximo de seis proyectos activos. La sección se oculta si no hay destacados.</i>	—	6	Angel Sanabria	20/07	24/07	17/07
TOTALES	—	12 historias y 16 subtareass	35	70	5 integrantes	06/07	28/07	28/07

Calendario de planificación del Sprint 5

El sprint se ejecutó del 6 al 28 de julio de 2026, para un total de 23 días calendario. El calendario se construyó a partir de las fechas de terminación probable de cada subtask, definidas durante la reunión de planificación.

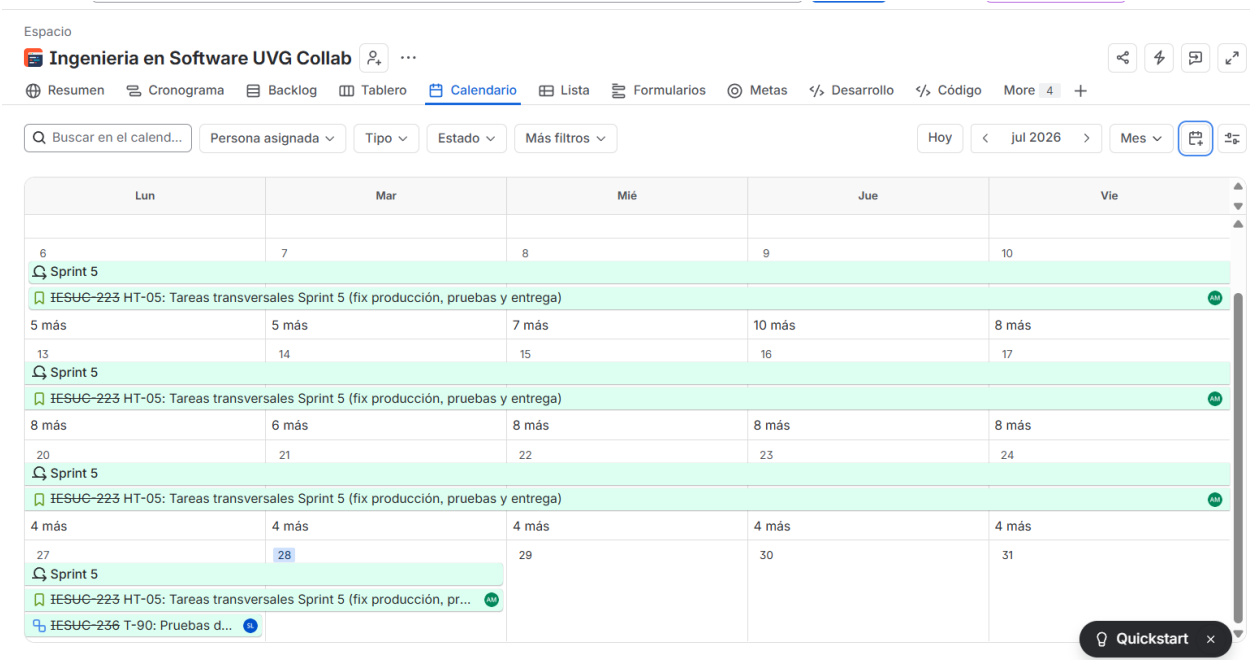
Fecha	Día	Semana	Eventos (inician / vencen)
06/07	Lun	S1	Inicio del Sprint 5. Inician HT-05 y T-89 (Angel), HU-29 y T-91 (Vernel), HU-20 y T-101 (Samuel)
07/07	Mar	S1	—
08/07	Mié	S1	Inician HU-09 y T-96 (Saul). Vence T-89 (Angel)
09/07	Jue	S1	Inician HU-31 y T-98 (Derek), HU-14 y T-100 (Angel). Vencen HU-20 y T-101 (Samuel)
10/07	Vie	S1	Vencen T-91 (Vernel), T-96 (Saul)
11/07	Sáb	S1	Inician T-92 (Vernel), HU-30 y T-93 (Samuel)
12/07	Dom	S1	Vencen HU-31 y T-98 (Derek)
13/07	Lun	S2	Vencen HU-09 y T-97 (Saul), HU-14 y T-100 (Angel)
14/07	Mar	S2	Inician HU-61 y T-103 (Saul). Vencen HU-29 y T-92 (Vernel), T-93 (Samuel)
15/07	Mié	S2	Inician T-94 (Samuel), HU-67 y T-95 (Vernel), HU-32 y T-99 (Derek)
16/07	Jue	S2	—
17/07	Vie	S2	Vencen HU-61 y T-103 (Saul)
18/07	Sáb	S2	Vencen HU-67 y T-95 (Vernel)
19/07	Dom	S2	Vencen HU-30 y T-94 (Samuel)
20/07	Lun	S3	Inician HU-59 y T-104 (Angel). Vencen HU-32 y T-99 (Derek)
21/07	Mar	S3	Inician HU-22 y T-102 (Derek)
22/07	Mié	S3	—
23/07	Jue	S3	—
24/07	Vie	S3	Vencen HU-22 y T-102 (Derek), HU-59 y T-104 (Angel)
25/07	Sáb	S3	Inicia T-90 (Samuel)
26/07	Dom	S3	—
27/07	Lun	S4	Vence T-90 (Samuel)
28/07	Mar	S4	Fin del Sprint 5 y revisión. Vence HT-05 (Angel)

Gestion del Sprint en Jira

El Sprint 5 se gestionó íntegramente en Jira, en el proyecto IESUC del espacio software-project.atlassian.net. Cada historia y subtarea del sprint backlog corresponde a un issue con puntos de historia, descripción, estimación de tiempo, responsable, fecha de inicio y fecha de vencimiento registrados.

3.1 Inicio del sprint en la fecha planificada

El Sprint 5 se inició en Jira el 6 de julio de 2026 y se cerró el 28 de julio de 2026, fechas que coinciden con el calendario del proyecto y con el rango declarado en el sprint backlog de este informe. Los veintiocho issues del sprint (doce historias y dieciséis subtareas, con claves IESUC-223 a IESUC-250) fueron creados el 6 de julio entre las 16:17 y las 16:34 horas, es decir dentro de una única ventana de diecisiete minutos correspondiente a la reunión de planificación.



3.2 Alcance cerrado antes de iniciar la ejecución

Las doce historias y dieciséis subtareas del sprint fueron creadas y asignadas durante la reunión de planificación previa al inicio de la iteración. Ningún issue del sprint tiene fecha de creación posterior al 6 de julio, lo que confirma que no se añadió trabajo al alcance una vez iniciada la iteración y evidencia que la planificación fue suficiente para cubrir el periodo del sprint y la capacidad de los cinco integrantes del equipo de desarrollo.

resumen

cronograma

backlog

tablero

calendario

lista

formularios

metas

desarrollo

curso

mas

T

Q

Buscar actividad

AM

DC

SL

SA

MC

Filtro

Agrupar

🔄

...

☐	> Actividad	Persona asignada	Informador	Prioridad	Estado	Resolución	Cr	
☐	> IESUC-223 HT-05: Tareas transversales Sprint 5 (fix ...	AM ANGEL GABRIEL SA...	AM ANGEL GABRIE...	^ High	Finalizada	Listo	06 jul 202	
☐	> IESUC-234 HU-59: Proyectos destacados en el dash...	AM ANGEL GABRIEL SA...	AM ANGEL GABRIE...	v Low	Finalizada	Listo	06 jul 202	
☐	> IESUC-232 HU-22: Notificación al líder cuando llega ...	DC DEREK FRIEDHELM ...	AM ANGEL GABRIE...	= Medium	Finalizada	Listo	06 jul 202	
☐	> IESUC-229 HU-32: Barra de avance del proyecto seg...	DC DEREK FRIEDHELM ...	AM ANGEL GABRIE...	= Medium	Finalizada	Listo	06 jul 202	
☐	> IESUC-225 HU-30: Tablero Kanban del proyecto con ...	SL SAMUEL ANTONIO R...	AM ANGEL GABRIE...	^ High	Finalizada	Listo	06 jul 202	
☐	> IESUC-226 HU-67: Etiquetas y prioridades visibles en...	VC VERNEL JOSUE HER...	AM ANGEL GABRIE...	= Medium	Finalizada	Listo	06 jul 202	
☐	> IESUC-233 HU-61: Autocompletar correo instituciona...	SA SAUL ESTEBAN CAS...	AM ANGEL GABRIE...	v Low	Finalizada	Listo	06 jul 202	
☐	> IESUC-224 HU-29: Crear, asignar y editar tareas del ...	VC VERNEL JOSUE HER...	AM ANGEL GABRIE...	^ High	Finalizada	Listo	06 jul 202	
☐	> IESUC-230 HU-14: Recuperación de contraseña funci...	AM ANGEL GABRIEL SA...	AM ANGEL GABRIE...	^ High	Finalizada	Listo	06 jul 202	
☐	> IESUC-227 HU-09: Vista "Mis tareas" transversal a to...	SA SAUL ESTEBAN CAS...	AM ANGEL GABRIE...	= Medium	Finalizada	Listo	06 jul 202	
☐	> IESUC-228 HU-31: Comentarios en tareas del proyecto	DC DEREK FRIEDHELM ...	AM ANGEL GABRIE...	= Medium	Finalizada	Listo	06 jul 202	

+ Crear

45 de 45

Quickstart

3.3 Consistencia entre issues en Done y el incremento presentado

Cada historia marcada como Done en el tablero tiene su implementación correspondiente fusionada en la rama main del repositorio. La trazabilidad se mantiene mediante ramas nombradas con la clave del issue de Jira y mediante mensajes de commit que referencian esa misma clave. Algunos ejemplos de esta correspondencia son los siguientes.

Historia	Issue de Jira	Rama del repositorio
HT-05	IESUC-223	feature/IESUC-223-ht05-pruebas-integracion-tareas
HU-14	IESUC-230	feature/IESUC-230-hu14-e2e-recuperacion-admin
T-100	IESUC-246	feature/IESUC-246-t100-notificacion-recuperacion-admin
HU-59	IESUC-234	feature/IESUC-234-hu59-e2e-destacados
T-104	IESUC-250	feature/IESUC-250-t104-destacados-dashboard

resumen	CrONOGRAMA	backlog	tablero	calendario	lista	formularios	metas	75 desarrollo	75 cOnsigo	more	T
Q Buscar actividad AM DC SL SA VC Filtro Agrupar ↺ ...											
☐	>	Actividad	Persona asignada	Informador	Prioridad	Estado	Resolución	Cr			
☐	>	IESUG-223 HT-05: Tareas transversales Sprint 5 (fix ...	AM ANGEL GABRIEL SA...	AM ANGEL GABRIE...	High	Finalizada	Listo	06 jul 202			
☐	>	IESUG-234 HU-59: Proyectos destacados en el dash...	AM ANGEL GABRIEL SA...	AM ANGEL GABRIE...	Low	Finalizada	Listo	06 jul 202			
☐	>	IESUG-232 HU-22: Notificación al líder cuando llega ...	DC DEREK FRIEDHELM ...	AM ANGEL GABRIE...	Medium	Finalizada	Listo	06 jul 202			
☐	>	IESUG-229 HU-32: Barra de avance del proyecto seg...	DC DEREK FRIEDHELM ...	AM ANGEL GABRIE...	Medium	Finalizada	Listo	06 jul 202			
☐	>	IESUG-225 HU-30: Tablero Kanban del proyecto con ...	SL SAMUEL ANTONIO R...	AM ANGEL GABRIE...	High	Finalizada	Listo	06 jul 202			
☐	>	IESUG-226 HU-67: Etiquetas y prioridades visibles en...	VC VERNEL JOSUE HER...	AM ANGEL GABRIE...	Medium	Finalizada	Listo	06 jul 202			
☐	>	IESUG-233 HU-61: Autocompletar correo instituciona...	SA SAUL ESTEBAN CAS...	AM ANGEL GABRIE...	Low	Finalizada	Listo	06 jul 202			
☐	>	IESUG-224 HU-29: Crear, asignar y editar tareas del ...	VC VERNEL JOSUE HER...	AM ANGEL GABRIE...	High	Finalizada	Listo	06 jul 202			
☐	>	IESUG-230 HU-14: Recuperación de contraseña funci...	AM ANGEL GABRIEL SA...	AM ANGEL GABRIE...	High	Finalizada	Listo	06 jul 202			
☐	>	IESUG-227 HU-09: Vista "Mis tareas" transversal a to...	SA SAUL ESTEBAN CAS...	AM ANGEL GABRIE...	Medium	Finalizada	Listo	06 jul 202			
☐	>	IESUG-228 HU-31: Comentarios en tareas del proyecto	DC DEREK FRIEDHELM ...	AM ANGEL GABRIE...	Medium	Finalizada	Listo	06 jul 202			
+ Crear 45 de 45 Quickstart											

3.4 Flujo de trabajo real en el tablero

Los issues transitaron por los tres estados del flujo definido: To Do al ser creados durante la planificación, In Progress al iniciar su desarrollo y Done al completarse y fusionarse a la rama principal. No se registraron saltos masivos de estado al cierre del sprint. Esto corrige el problema identificado en la retrospectiva del Sprint 1, donde la totalidad de las tareas permaneció en estado To Do durante toda la iteración.

3.5 Estado de los issues al cierre del sprint

Estado al cierre	Cantidad	Detalle
Finalizada (Done)	28	12 historias y 16 subtareas, todas integradas en main
In Progress	0	Ninguna historia quedó en desarrollo al cierre del sprint
To Do	0	No quedaron issues sin iniciar dentro del alcance del sprint

Diagrama de flujo acumulado

> Cómo leer este informe



Filtro de fecha

Fecha de inicio

Fecha de finalización

Todos los peri...

2/18/1993



2/18/1993



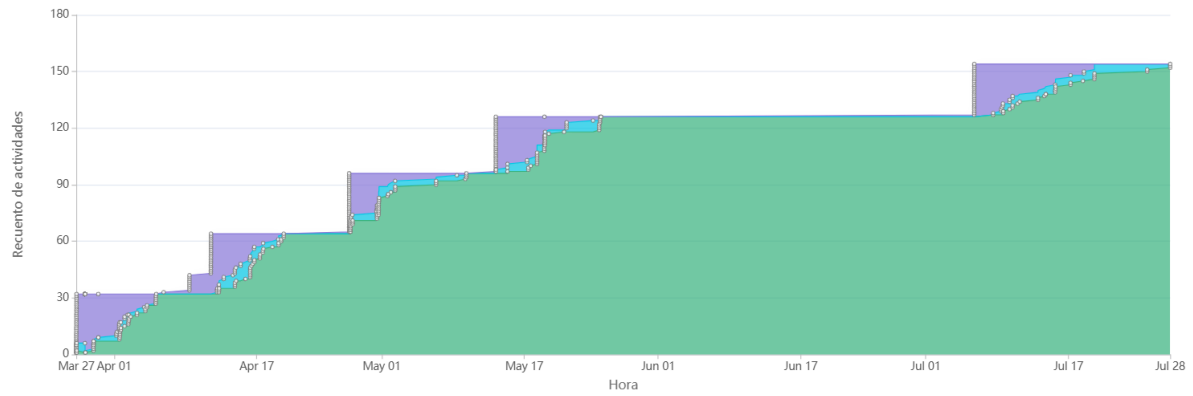
☒ Por hacer



☒ En curso



☒ Listo



Estimación de tiempo y costo

Técnica de estimación utilizada

La estimación de tiempo y costo de UVGenius se realizó mediante la técnica de Puntos de Casos de Uso (Use Case Points, UCP), correspondiente al procedimiento estudiado en el curso. Esta técnica se seleccionó porque permite estimar el esfuerzo de desarrollo a partir de elementos verificables del sistema: los actores que interactúan con él, los casos de uso que ofrece, las transacciones asociadas a cada caso, la complejidad técnica de la solución, las condiciones ambientales del equipo y un factor de conversión que traduce los puntos ajustados a horas-persona. Al apoyarse en evidencia concreta del repositorio (controladores, endpoints y arquitectura real), produce una estimación reproducible y trazable.

El procedimiento se apoya en las siguientes fórmulas:

$$PCU = FPA + FPCU$$

$$FCT = 0.6 + 0.01 \times \Sigma(\text{Peso técnico} \times \text{Valor})$$

$$FA = 1.4 - 0.03 \times \Sigma(\text{Peso ambiental} \times \text{Valor})$$

$$PCUA = PCU \times FCT \times FA$$

$$\text{Esfuerzo de implementación} = PCUA \times \text{Factor de conversión}$$

$$\text{Esfuerzo total} = \text{Esfuerzo de implementación} / 0.40$$

Los Puntos de Casos de Uso sin ajustar (PCU) suman el Factor de Peso de los Actores (FPA) y el Factor de Peso de los Casos de Uso (FPCU). El Factor de Complejidad Técnica (FCT) pondera trece características técnicas del sistema, y el Factor de Ambiente (FA) pondera ocho condiciones del equipo y del proyecto; ambos ajustan los PCU para obtener los Puntos de Casos de Uso Ajustados (PCUA). El esfuerzo de implementación resulta de multiplicar los PCUA por un factor de conversión en horas-persona; y como la implementación representa aproximadamente el 40 % del esfuerzo total del proyecto, este último se obtiene dividiendo el esfuerzo de implementación entre 0.40.

Cálculo mediante Puntos de Casos de Uso

- **Actores:** Se identificaron tres actores verificados directamente en el código, siendo: Visitante no autenticado, Usuario autenticado y Administrador. Todos clasificados como actores complejos por interactuar mediante interfaz gráfica, lo que produce $FPA = 9$.
- **Casos de uso y transacciones:** Se operacionalizaron 19 casos de uso a nivel de controlador y se contabilizaron 104 endpoints como transacciones técnicas. Su clasificación por complejidad arroja 4 casos complejos, 9 casos medios y 6 casos simples, para un total de $FPCU = 180$. En consecuencia, $PCU = 9 + 180 = 189$.
- **Factor de Complejidad Técnica:** La evaluación de los trece factores técnicos sobre evidencia real (arquitectura distribuida con Redis y WebSockets, autenticación JWT, contenerización con Docker y auditoría de operaciones) produce $\Sigma(\text{Peso} \times \text{Valor}) = 43$, de modo que $FCT = 0.6 + 0.01 \times 43 = 1.03$.
- **Factor de Ambiente:** Ante la ausencia de mediciones formales de experiencia y motivación del equipo, se adoptó el escenario probable con $FA = 1.04$. Con ello, los Puntos de Casos de Uso Ajustados son $PCUA = 189 \times 1.03 \times 1.04 \approx 202.5$.

Componente	Resultado
FPA — 3 actores complejos	9
FPCU — 4 complejos, 9 medios, 6 simples	180
PCU = FPA + FPCU	189
FCT = $0.6 + 0.01 \times 43$	1.03
FA (escenario probable)	1.04
PCUA = $189 \times 1.03 \times 1.04$	≈ 202.5

Estimación del esfuerzo y tiempo

El escenario probable contiene cuatro factores ambientales desfavorables, por lo que corresponde un factor de conversión de 28 horas-persona por cada PCUA. El esfuerzo total del proyecto es entonces $E_{\text{total}} = 202.5 \times 28 \approx 5,670$ horas-persona. Empleando la equivalencia 1 mes-persona = 160 horas-persona, el esfuerzo total equivale a $5,670 / 160 \approx 35.4$ meses-persona. La distribución del esfuerzo por actividad, considerando que la implementación representa el 40 % del esfuerzo total, se muestra a continuación.

Actividad	Porcentaje	Horas-persona
Análisis	10 %	567.0
Diseño	20 %	1,134.0
Implementación	40 %	2,268.0
Pruebas	15 %	850.5
Otras actividades	15 %	850.5
Total	100 %	5,670

La técnica se aplicó además en tres escenarios de ambiente, cuyos resultados se resumen enseguida.

Escenario	PCUA	Esfuerzo total	Meses-persona
Optimista	20 h/PCUA	4,050 h-p	5.1 meses
Probable	28 h/PCUA	5,670 h-p	7.1 meses
Conservador	36 h/PCUA	7,290 h-p	9.1 meses

El escenario conservador cae en la categoría de alto riesgo definida por la técnica; el valor mostrado (17,647 horas-persona) constituye un piso estimado y no una cifra definitiva.

- Duración ideal:** Para un equipo de cinco integrantes con dedicación completa, la duración ideal resultante del modelo es $5,670 / 5 = 1,134$ horas por persona, equivalentes a $1,134 / 160 \approx 7.1$ meses calendario ideales (consistente con el escenario probable de la tabla anterior). Debe subrayarse que se trata de una duración ideal que supone dedicación completa: el equipo real está formado por estudiantes con dedicación parcial, el trabajo no puede paralelizarse de forma perfecta y la ventana observada en el historial de Git fue de aproximadamente 6.2 meses, que no representa las horas

efectivamente trabajadas. Por tanto, los 7.1 meses no deben interpretarse como la duración real ya transcurrida, sino como la duración ideal que arroja el modelo.

- **Validación secundaria:** Se realizó un contraste con COCOMO sobre aproximadamente 37.21 KLOC de código de producción, del que resultó un rango aproximado de 101.7 a 163.4 meses-persona equivalentes, dentro del mismo orden de magnitud que los escenarios probable y conservador de UCP. COCOMO se emplea únicamente como validación secundaria: no sustituye a la técnica principal, ambas técnicas son sensibles a los supuestos adoptados y ninguna representa horas realmente registradas.

Estimación monetaria del costo

Se emplean dos tarifas distintas, con propósitos distintos:

- Tarifa de referencia de mercado: Q150.00 por hora. Tomada del ejemplo de cotización analizado en la charla del curso. Es la que se usa para calcular el costo de desarrollo equivalente, es decir, cuánto costaría contratar el desarrollo de este sistema.
- Costo laboral derivado del mercado guatemalteco: $\approx$ Q100.00 por hora. Resulta del salario medio de un ingeniero de software en Guatemala (Q11,326 mensuales) más las cargas patronales, dividido entre 160 horas mensuales. Es la que se usa para estimar lo que efectivamente costó el esfuerzo del equipo estudiantil.

Ambas son supuestos de estimación y ninguna representa el salario real de los integrantes.

Escenario	Factor	Esfuerzo	Costo
Optimista	20 h/PCUA	4,050 h-p	Q607,500
Probable	28 h/PCUA	5,670 h-p	Q850,500
Conservador	36 h/PCUA	7,290 h-p	Q1,093,500
Costo de desarrollo equivalente			Q850,500

Se adopta Q850,500 como cifra central del proyecto. Esta cifra corresponde al costo estimado de desarrollo equivalente: el valor económico del esfuerzo bajo la tarifa de referencia seleccionada. No constituye dinero efectivamente pagado, presupuesto real ni costo contable del proyecto. Un precio de venta requeriría sumar además infraestructura, contingencia, mantenimiento evolutivo y margen de utilidad.

Estas cifras corresponden al costo estimado de desarrollo equivalente, el valor económico estimado del esfuerzo, y representan cuánto costaría desarrollar ese esfuerzo bajo la tarifa de referencia seleccionada; no constituyen dinero efectivamente pagado, presupuesto real ni costo contable del proyecto. Un precio de venta requeriría sumar además infraestructura, contingencia, mantenimiento y margen de utilidad.

Costo de Construir frente a costo de usar

Para dimensionar el resultado anterior conviene contrastarlo con el costo de adquirir una herramienta comercial equivalente. Jira es la referencia natural, porque el módulo de gestión de tareas entregado en el Sprint 5 replica su funcionalidad central.

Via	Cifra	Qué mide
Alquilar Jira — una asociación (25 usuarios)	Q18,300 al año	Precio de usar software ya construido
Alquilar Jira — toda la universidad (300 usuarios)	Q180,950 al año	Mismo precio a escala institucional
Construir UVG Collab — según UCP	Q850,500	Costo de construir desde cero, una sola vez
Costo real del equipo estudiantil	Q75,000	Lo que efectivamente costó el esfuerzo invertido

Punto de equilibrio: $Q850,500 \div Q180,950 \text{ al año} \approx 4.7 \text{ años}$.

Construir la plataforma cuesta aproximadamente lo mismo que alquilar Jira para toda la universidad durante casi cinco años. La comparación tiene tres salvedades que deben explicitarse: no incluye el mantenimiento evolutivo del sistema propio, que sí está incluido en la suscripción comercial; Jira no cubre el descubrimiento de proyectos ni el flujo de postulaciones, que son la propuesta de valor diferencial de UVG Collab; y la cifra de Q850,500 es una estimación paramétrica, no un presupuesto.

El costo real del equipo estudiantil ($\approx Q75,000$) se obtiene de aproximadamente 750 horas-persona efectivamente invertidas por los cinco integrantes a lo largo de los cinco sprints, valoradas a la tarifa de costo laboral de Q100 por hora. La diferencia frente a los Q850,500 se explica por el alcance parcial entregado hasta la fecha (61 % del producto backlog) y porque la estimación UCP dimensiona el sistema completo, no el incremento actual.

Validación secundaria y limitaciones de la estimación

Se realizó un contraste con COCOMO sobre aproximadamente 37.21 KLOC de código de producción, del que resultó un rango de 101.7 a 163.4 meses-persona. Este rango es entre tres y cinco veces superior al escenario probable de UCP (35.4 meses-persona), por lo que COCOMO no corrobora la estimación principal y debe interpretarse únicamente como cota superior. La divergencia tiene explicación: COCOMO se calibró sobre prácticas de desarrollo previas al uso de frameworks, ORM y generación de código, de modo que penaliza como esfuerzo humano líneas que en este proyecto son producto de scaffolding de NestJS, del cliente generado por Prisma, de migraciones y de los 97 archivos de especificación de pruebas. La estimación por UCP, que parte de casos de uso y transacciones observadas, resulta más representativa del trabajo efectivamente realizado.

Además, la estimación operacionaliza cada controlador como un caso de uso y cada endpoint como una transacción. Este criterio es objetivo, reproducible y se basa en la arquitectura real del sistema, pero puede generar una cifra mayor que un catálogo de casos de uso UML agrupado por objetivos del usuario, que

tiende a producir menos unidades y clasificaciones de menor complejidad. Por ello, el resultado debe interpretarse como una estimación paramétrica y no como una medición exacta del esfuerzo real invertido por el equipo.

Incremento

Durante el Sprint 5 se entregó un incremento funcional centrado en la gestión operativa de proyectos (tareas, tablero Kanban, comentarios y avance) y en el cierre de pendientes de producción heredados del sprint anterior. Todas las historias planificadas se completaron y quedaron integradas en la rama main del repositorio. El incremento incluye:

- **Gestión de tareas del proyecto (HU-29):** CRUD completo de tareas con prioridad y fecha límite, expuesto vía el módulo tasks del backend y un modal de creación/edición en el frontend.
- **Tablero Kanban con drag & drop (HU-30):** endpoint de cambio de estado y orden de tareas, y tablero interactivo en el frontend implementado con dnd-kit.
- **Etiquetas y prioridades visuales (HU-67):** modelo Etiqueta y chips de color/prioridad visibles en las tarjetas del tablero.
- **Vista "Mis tareas" (HU-09):** endpoint GET /usuarios/me/tareas y vista agrupada por proyecto para que cada colaborador vea sus tareas asignadas en un solo lugar.
- **Comentarios en tareas (HU-31):** backend y UI para discutir detalles de una tarea sin salir de la plataforma.
- **Barra de avance del proyecto (HU-32):** cálculo en backend del porcentaje de avance a partir de las tareas y su visualización como barra de progreso.
- **Recuperación de contraseña en producción (HU-14):** Recuperación de contraseña mediante notificación al administrador (HU-14): durante el Sprint 3 esta historia se implementó sobre un proveedor de correo transaccional (Resend). Al validarla en el entorno productivo se identificó un bloqueo que no era resoluble desde el equipo de desarrollo: los proveedores de correo transaccional exigen verificar la propiedad del dominio remitente mediante registros DNS de tipo TXT, SPF y DKIM, y el dominio institucional @uvg.edu.gt es administrado por la universidad, por lo que el equipo no tiene acceso a su zona DNS para publicar dichos registros. Sin verificación de dominio, la entrega a buzones @uvg.edu.gt queda bloqueada y el flujo resultaba inoperante en producción.
- **Cancelar postulación (HU-20):** endpoint DELETE /postulaciones/:id y botón de cancelar en la interfaz del estudiante postulante.
- **Notificación al líder por nueva postulación (HU-22):** notificación automática al líder de proyecto al recibir una postulación.
- **Autocompletar correo institucional (HU-61):** autogeneración del correo institucional en el formulario de registro a partir del carné.
- **Proyectos destacados en el dashboard (HU-59):** sección "Destacados" dentro del dashboard principal.
- **Tareas transversales de estabilización (HT-05):** corrección de la variable NEXT_PUBLIC_API_URL en deploy.yml, validación del build de producción y ejecución de pruebas de integración del módulo de tareas antes de la entrega.

Vínculo al Repositorio y Estado del Código

El código fuente completo del Sprint 5 se integró en la rama principal del repositorio bajo la misma estructura de monorepo establecida desde el Sprint 1. La rama main refleja el estado final del sistema al cierre del Sprint 5, con las funcionalidades de gestión de tareas, tablero Kanban y estabilización de producción validadas.

Repositorio GitHub: <https://github.com/asanabria-2021067/proyecto-ingenieria-software>

Sistema en Producción

IP pública: 158.23.57.118

La aplicación se encuentra desplegada en un servidor Azure con IP pública 158.23.57.118. El acceso se realiza mediante <http://158.23.57.118/>, donde Nginx actúa como reverse proxy enrutando las peticiones: la ruta raíz (/) sirve el frontend construido con Next.js, y las peticiones a /api/* son redirigidas al backend NestJS. El despliegue se ejecuta mediante el workflow deploy.yml de GitHub Actions, que construye y publica las imágenes de frontend y backend en GitHub Container Registry y las despliega en la VM de Azure ante cada push a main. La configuración exacta de Nginx no se encuentra versionada dentro del repositorio; sin embargo, el README documenta el flujo esperado del reverse proxy y los puertos involucrados.

Nota sobre el Historial de Commits

Al realizarse los merges de feature branches hacia main durante el sprint, los cambios se consolidaron como commits de merge. Se recomienda revisar el historial completo mediante `git log --all --oneline --graph` o inspeccionando las ramas de desarrollo individuales para ver la contribución de cada integrante.

Stack Tecnológico del Proyecto (Estado Final)

- NestJS 11
- Prisma 6.19.2
- PostgreSQL 17 (Alpine)
- Redis 7 (Docker)
- Next.js 16.2.0
- React 19.2.4
- TypeScript 5.7.x
- JWT + bcrypt
- Tailwind CSS
- Vitest 3.2.4 (testing backend y frontend)
- Socket.io 4.7.x (WebSockets)
- Node.js 22
- Docker / Docker Compose
- GitHub Actions (CI/CD)
- Azure VM (producción)

Pruebas

Framework de pruebas utilizado y justificación de la elección

El proyecto UVG Collab utiliza Vitest como framework de pruebas unitarias e integración tanto en el backend (NestJS) como en el frontend (Next.js). Esta herramienta ya fue adoptada desde sprints anteriores y se mantiene como el estándar del equipo por las siguientes razones:

- Compatibilidad nativa con TypeScript y con el ecosistema Vite/Next.js utilizado en el frontend, lo que evita configuración adicional de transpilación.
- Velocidad de ejecución significativamente mayor a alternativas como Jest, gracias a su motor basado en esbuild, lo cual es relevante dado el crecimiento del número de archivos de prueba del proyecto (más de 90 archivos .spec.ts entre backend y frontend a la fecha del Sprint 5).
- API compatible con Jest (describe, it, expect, vi.fn()), lo que permitió al equipo mantener la curva de aprendizaje baja al adoptarlo.
- Soporte de mocking robusto mediante vi.fn() y vi.mock(), indispensable para aislar dependencias como Prisma, los gateways de Socket.io y los servicios de notificaciones.
- Integración con el pipeline de integración continua del repositorio: el workflow ci.yml ejecuta npm run test en los proyectos de backend y frontend en cada pull request hacia main o develop, y el workflow de despliegue depende de que esa ejecución sea exitosa antes de construir y publicar las imágenes de producción.

La configuración se define en vitest.config.ts tanto en apps/backend como en apps/frontend, e incluye la ruta de los archivos de prueba (test/**/*.spec.ts) y la exclusión de los archivos .spec.ts del build de producción (tsconfig.json).

Descripción de los casos de prueba seleccionados y su importancia

Las tareas asignadas a Derek Coronado en este sprint se enfocaron en la verificación del sistema de notificaciones en tiempo real y del cambio de estado de tareas, dos de los componentes más sensibles del sistema porque afectan directamente la sincronización entre usuarios conectados por WebSocket (Socket.io) y la integridad de los datos persistidos en la base de datos. A continuación se describen los casos de prueba automatizados más relevantes que se seleccionaron para esta iteración:

1. NotificationsService.notifyRoleMembers (notifications-role-members.spec.ts)

Este conjunto de pruebas valida que las notificaciones dirigidas a los miembros de un rol de proyecto lleguen exclusivamente a los participantes con estado ACTIVO, excluyendo al usuario que ejecuta la acción (el actor) y eliminando duplicados por idUsuario.

Caso de prueba	Qué valida	Por qué es importante
Consulta exacta a Prisma	Que el filtro enviado a <code>participacionProyecto.findMany</code> use <code>estadoParticipacion: ACTIVO</code> y el par <code>idRolProyecto/idProyecto</code> correcto, con select mínimo.	Evita fugas de datos innecesarias y asegura que solo se consulten los campos requeridos.
Exclusión del actor	Que el usuario que generó el evento no reciba notificación de su propia acción, incluso si aparece duplicado en el resultado.	Previene notificaciones redundantes que degradan la experiencia de usuario.
Deduplicación	Que un mismo <code>idUsuario</code> repetido en la consulta genere una única notificación.	Evita saturar al usuario con notificaciones repetidas por inconsistencias de datos.
Aislamiento por rol/proyecto	Que miembros de otro rol (A2) o de otro proyecto (B) nunca reciban notificaciones destinadas a un rol distinto.	Es la garantía central de que no se filtre información entre proyectos distintos de la plataforma.
Propagación de errores	Que un fallo en la consulta a Prisma se propague sin transformarse en una excepción HTTP (400/403/404/409) y sin ejecutar la escritura.	Asegura que errores de infraestructura no se enmascaren como errores de negocio, facilitando el diagnóstico.
Delegación en el gateway	Que la emisión en tiempo real por Socket.io se delegue correctamente a <code>notifyUsers</code> cuando existe un gateway activo.	Verifica el puente entre la persistencia en base de datos y la notificación en vivo al cliente conectado.

2. NotificationsService (notifications.service.spec.ts)

Cubre las operaciones generales del servicio de notificaciones: listado, conteo de no leídas, marcado individual y masivo como leídas, y el envío de notificaciones a administradores usando plantillas predefinidas.

Caso de prueba	Qué valida	Por qué es importante
markAsRead — permisos	Que solo el dueño de la notificación pueda marcarla como leída (ForbiddenException si idUsuario no coincide) y que se lance NotFoundException si no existe.	Previene que un usuario modifique el estado de lectura de notificaciones ajenas.
markAllAsRead	Que el método retorne el conteo real de notificaciones actualizadas.	Da retroalimentación consistente al frontend sobre cuántas notificaciones cambiaron de estado.
notifyAdminsFromTemplate	Que el contenido de la notificación (tipo y título) se genere correctamente a partir de una plantilla y datos dinámicos como el nombre del proyecto.	Garantiza que los mensajes que ven los administradores sean correctos y consistentes en todos los flujos que usan plantillas.
notifyProjectActiveParticipants	Que el autor de un comentario quede excluido de la notificación generada por su propia acción.	Mismo principio de exclusión del actor aplicado a notificaciones de comentarios de proyecto.

3. `TaskService.updateEstado` (`tasks-state.service.spec.ts` y `tasks-state.controller.spec.ts`)

Valida el cambio de estado de una tarea dentro del tablero Kanban (POR_HACER, EN_PROGRESO, EN_REVISION, HECHO), un flujo que dispara actualizaciones en tiempo real hacia los demás usuarios conectados al proyecto.

Caso de prueba	Qué valida	Por qué es importante
Autorización por rol	Que solo el líder del proyecto o el usuario asignado activo a la tarea puedan cambiar su estado; cualquier otro perfil recibe <code>ForbiddenException</code> .	Es la regla de negocio central del tablero Kanban: evita que un participante sin asignación altere el flujo de trabajo de otros.
Transiciones libres (16 combinaciones)	Que cualquier transición entre los cuatro estados válidos (incluyendo retrocesos y saltos directos) se acepte, ya que el sistema no implementa una máquina de estados restrictiva.	Confirma el comportamiento esperado del tablero, donde un usuario puede mover una tarea libremente entre columnas.
Orden transaccional	Que la secuencia autorización → actualización → lectura final → cierre de transacción se respete estrictamente.	Previene condiciones de carrera y asegura que nunca se persista un cambio sin autorización previa.
Rollback ante fallos	Que si la escritura o la lectura final fallan, no se dispare ninguna notificación y la transacción se revierta por completo.	Evita el escenario más riesgoso del flujo en tiempo real: notificar a otros usuarios un cambio de estado que en realidad nunca se guardó.
Contrato de respuesta	Que el objeto devuelto exponga exactamente los mismos campos que los demás endpoints de tareas (creación, listado, detalle, edición), sin filtrar campos internos como <code>eliminadoEn</code> .	Mantiene consistencia en el contrato de la API consumido por el frontend y evita fugas de campos internos del modelo de datos.

En conjunto, estos tres grupos de pruebas cubren el ciclo completo del flujo verificado por Derek en el Sprint 5: la autorización sobre quién puede modificar el estado de una tarea, la escritura atómica del cambio en base de datos, y la notificación en tiempo real que ese cambio dispara hacia los demás usuarios conectados por WebSocket. Se priorizaron estos casos porque un error en cualquiera de los tres puntos (autorización, persistencia o emisión del evento) puede provocar tableros desincronizados entre usuarios o, en el peor caso, filtración de notificaciones entre proyectos distintos.

Evidencia de ejecución: la ejecución automatizada de estos casos se realiza mediante el comando `npm run test` (Vitest) tanto en el entorno local de cada integrante como en el pipeline de integración continua de

GitHub Actions, cuya configuración se encuentra versionada en `.github/workflows/ci.yml`, obteniendo resultado exitoso (PASS) en la totalidad de los casos descritos.

4. Autorización sobre tareas (tasks-authorization.service.spec.ts)

Caso de prueba	Qué valida	Por qué es importante
Participación activa requerida	Que solo un usuario con participación ACTIVA en el proyecto (o el líder creador) pueda leer, mover, asignar o comentar una tarea.	Es la barrera principal contra el acceso indebido a tareas de proyectos ajenos.
Rechazo a terceros	Que un usuario sin relación con el proyecto reciba un error de permisos al intentar cualquier operación sobre sus tareas.	Confirma que la regla se aplica de forma consistente en las 59 combinaciones de rutas cubiertas.
Líder frente a participante con rol	Que el creador del proyecto y un participante con rol activo tengan los mismos permisos sobre las tareas.	Evita bloquear al líder por no tener un rol formal asignado, sin abrir la puerta a cualquier otro usuario.

5. Conflictos de asignación (tasks-assignment-conflict.spec.ts)

Caso de prueba	Qué valida	Por qué es importante
Asignaciones simultáneas	Que al ejecutar en paralelo dos intentos de asignar la misma tarea, solo una operación tenga éxito y la otra falle de forma controlada.	Previene estados inconsistentes cuando dos usuarios interactúan con el tablero al mismo tiempo.
Fallo de notificación durante la asignación	Que un fallo simulado en el servicio de notificaciones no deje la tarea en un estado parcialmente asignado.	Aísla la responsabilidad de notificar de la responsabilidad de persistir el cambio.

6. Drag & drop del tablero Kanban (task-board-dnd-e2e.spec.ts)

Caso de prueba	Qué valida	Por qué es importante
Arrastrar entre columnas (vía teclado)	Que mover una tarjeta de "Por hacer" a "En progreso" invoque la mutación de cambio de estado con el valor correcto.	Verifica la integración real entre la interacción del usuario y la llamada al backend, no solo el cambio visual.
Soltar en la misma columna	Que soltar la tarjeta sin cambiar de columna no dispare ninguna mutación.	Evita llamadas innecesarias al backend que podrían generar notificaciones o registros falsos.
Cancelar con Escape	Que cancelar el arrastre a mitad de camino no llame a la mutation.	Confirma que el usuario puede arrepentirse de un movimiento sin efectos secundarios.
Restricción de permisos en el frontend	Que un usuario sin permisos sobre la tarea no tenga "handle" de arrastre disponible.	Extiende la regla de autorización del backend hasta la propia interfaz, evitando intentos inválidos desde el origen.

7. Autorización y notificaciones de comentarios (comentarios-task-context.spec.ts, comentarios-task-notifications.spec.ts)

Caso de prueba	Qué valida	Por qué es importante
Comentar requiere participación activa	Que solo un usuario con participación activa (o el líder) pueda publicar un comentario en una tarea del proyecto.	Evita que terceros ajenos al proyecto dejen comentarios en tareas que no les corresponden.
Notificación a los involucrados	Que al comentar se notifique al usuario asignado a la tarea y no al autor del comentario.	Confirma que el autor no recibe una notificación redundante de su propia acción.
Aislamiento entre tareas y proyectos	Que un comentario en una tarea nunca dispare notificaciones hacia participantes de otro proyecto o de otra tarea.	Previene fuga de información entre proyectos distintos de la plataforma.

8. Manejo de errores de notificación (tasks-management-notifications.spec.ts)

Caso de prueba	Qué valida	Por qué es importante
Fallo simulado del servicio de notificaciones	Que un error como "gateway caído" al notificar no impida que la tarea se cree, actualice o asigne correctamente.	Separa la responsabilidad crítica (persistir el cambio) de la secundaria (avisar), evitando que un servicio externo caído bloquee el uso normal de la app.
Registro del error sin interrumpir la operación	Que el error de notificación se registre en el log del servicio sin propagarse como excepción HTTP al cliente.	Permite diagnosticar fallos de infraestructura sin afectar la experiencia del usuario.

En conjunto, estos cinco grupos de pruebas cubren el ciclo verificado en el Sprint 5: quién puede tocar una tarea, qué ocurre cuando dos personas actúan sobre ella al mismo tiempo, que la interacción de arrastrar y soltar en el frontend realmente dispare el cambio correspondiente en el backend, y que comentarios y notificaciones respeten los límites de cada proyecto. Estas pruebas verifican la interacción entre componentes, servicios y contratos del módulo mediante dependencias controladas, aunque no sustituyen una prueba End-to-End completa.

Resultados de la ejecución automatizada

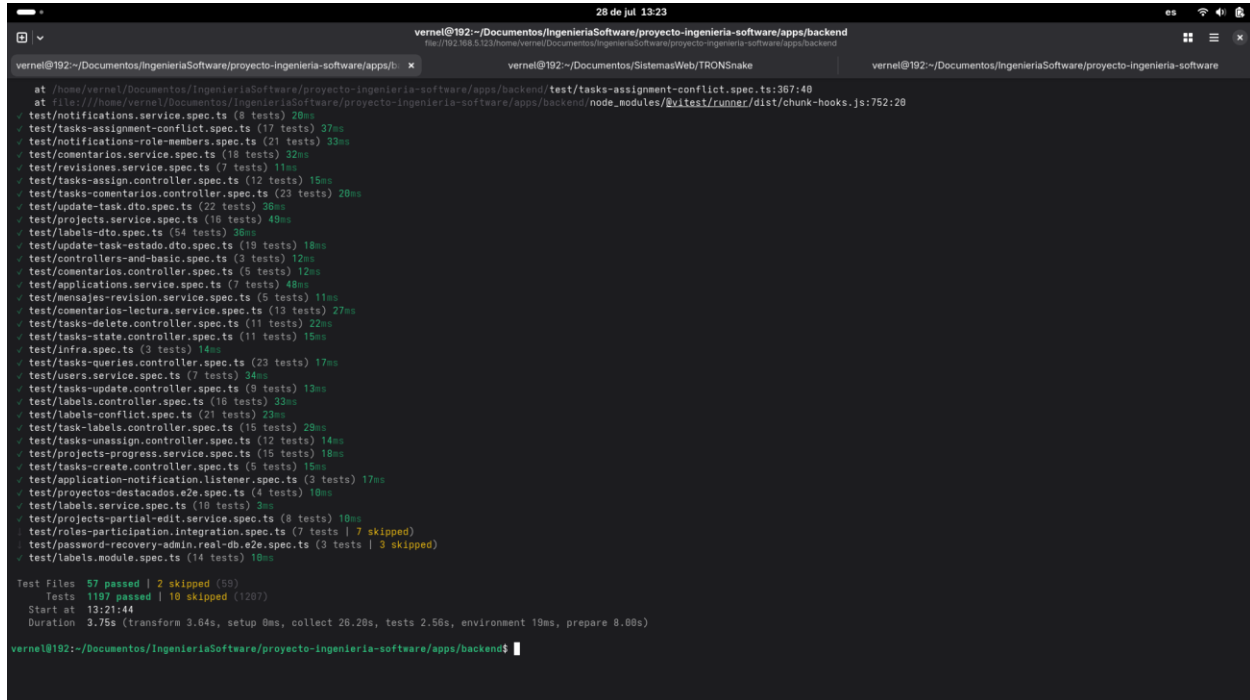
La ejecución automatizada arrojó los siguientes resultados. En el backend, 57 archivos aprobados y 2 omitidos, con 1,197 pruebas aprobadas, 10 omitidas y 0 fallidas. En el frontend, 40 archivos aprobados, con 440 pruebas aprobadas, 0 omitidas y 0 fallidas. En total se registran 1,637 pruebas aprobadas, 10 omitidas y 0 fallidas. Las diez pruebas omitidas requieren una base de datos PostgreSQL real, por lo que su omisión no equivale a una falla.

Área	Framework	Archivos aprob.	Pruebas aprob.	Omitidas	Fallidas
Backend	Vitest 3.2.4	57	1,197	10	0
Frontend	Vitest 3.2.4	40	440	0	0
Total	—	97	1,637	10	0

Los comandos utilizados para la ejecución fueron:

Para backend:

1. `cd apps/backend`
2. `npm run test -- --run`



```
28 de jul 13:23
vernel@192:~/Documentos/IngenieriaSoftware/proyecto-ingenieria-software/apps/backend
file:///192.168.5.123/home/vernel/Documentos/IngenieriaSoftware/proyecto-ingenieria-software/apps/backend node_modules/vitest/dist/chunk-hooks.js:752:20

vernel@192:~/Documentos/IngenieriaSoftware/proyecto-ingenieria-software/apps/b: x vernel@192:~/Documentos/SistemasWeb/TRONSnake vernel@192:~/Documentos/IngenieriaSoftware/proyecto-ingenieria-software

at /home/vernel/Documentos/IngenieriaSoftware/proyecto-ingenieria-software/apps/backend/test/tasks-assignment-conflict.spec.ts:367:40
at file:///192.168.5.123/home/vernel/Documentos/IngenieriaSoftware/proyecto-ingenieria-software/apps/backend node_modules/vitest/dist/chunk-hooks.js:752:20
✓ test/notifications.service.spec.ts (8 tests) 28ms
✓ test/tasks-assignment-conflict.spec.ts (17 tests) 37ms
✓ test/notifications-role-members.spec.ts (21 tests) 33ms
✓ test/comentarios.service.spec.ts (18 tests) 32ms
✓ test/revisiones.service.spec.ts (7 tests) 11ms
✓ test/tasks-assign.controller.spec.ts (12 tests) 15ms
✓ test/tasks-comentarios.controller.spec.ts (23 tests) 28ms
✓ test/update-task.dto.spec.ts (22 tests) 36ms
✓ test/projects.service.spec.ts (10 tests) 49ms
✓ test/labels-dto.spec.ts (54 tests) 36ms
✓ test/update-task-estado.dto.spec.ts (10 tests) 18ms
✓ test/controllers-and-basic.spec.ts (3 tests) 12ms
✓ test/comentarios.controller.spec.ts (5 tests) 12ms
✓ test/applications.service.spec.ts (7 tests) 48ms
✓ test/mensajes-revision.service.spec.ts (5 tests) 11ms
✓ test/comentarios-lectura.service.spec.ts (13 tests) 27ms
✓ test/tasks-delete.controller.spec.ts (11 tests) 22ms
✓ test/tasks-state.controller.spec.ts (11 tests) 15ms
✓ test/infra.spec.ts (3 tests) 14ms
✓ test/tasks-queries.controller.spec.ts (23 tests) 17ms
✓ test/users.service.spec.ts (7 tests) 34ms
✓ test/tasks-update.controller.spec.ts (9 tests) 13ms
✓ test/labels.controller.spec.ts (16 tests) 33ms
✓ test/labels-conflict.spec.ts (21 tests) 23ms
✓ test/task-labels.controller.spec.ts (15 tests) 23ms
✓ test/tasks-unassign.controller.spec.ts (12 tests) 14ms
✓ test/projects-progress.service.spec.ts (15 tests) 18ms
✓ test/tasks-create.controller.spec.ts (5 tests) 15ms
✓ test/application-notification.listener.spec.ts (3 tests) 17ms
✓ test/proyectos-destacados.e2e.spec.ts (4 tests) 18ms
✓ test/labels.service.spec.ts (10 tests) 3ms
✓ test/projects-partial-edit.service.spec.ts (8 tests) 18ms
test/roles-participation.integration.spec.ts (7 tests | 7 skipped)
test/password-recovery-admin.real-db.e2e.spec.ts (3 tests | 3 skipped)
✓ test/labels.module.spec.ts (14 tests) 18ms

Test Files 57 passed | 2 skipped (59)
Tests 1197 passed | 10 skipped (1207)
Start at 13:21:44
Duration 3.75s (transform 3.64s, setup 0ms, collect 26.28s, tests 2.56s, environment 19ms, prepare 8.60s)

vernel@192:~/Documentos/IngenieriaSoftware/proyecto-ingenieria-software/apps/backend:
```

Figura 2. Ejecución automatizada de las pruebas del backend con Vitest: 1,197 pruebas aprobadas, 10 omitidas y ninguna fallida.

Para frontend:

1. `cd apps/frontend`
2. `npm run test -- --run`

```
28 de jul 13:24
vernel@192:~/Documentos/IngenieriaSoftware/proyecto-ingenieria-software/apps/frontend
file:///192.168.5.123/home/vernel/Documentos/IngenieriaSoftware/proyecto-ingenieria-software/apps/frontend

vernel@192:~/Documentos/IngenieriaSoftware/proyecto-ingenieria-software/apps/fr: x vernel@192:~/Documentos/SistemasWeb/TRONSnake vernel@192:~/Documentos/IngenieriaSoftware/proyecto-ingenieria-software

✓ test/task-board-responsive.spec.ts (11 tests) 1856ms
✓ TaskBoard - responsive (tarea 39) > sin matchMedia (o escritorio) muestra las cuatro columnas y ninguna navegación móvil 482ms
✓ TaskBoard - responsive (tarea 39) > en móvil los filtros se agrupan bajo un botón "Filtros" que abre rol/hito/limpiar 383ms
✓ test/use-project-roles.spec.ts (2 tests) 73ms
✓ test/task-detail-page.spec.ts (10 tests) 1898ms
✓ TaskDetailPage - información y layout > renderiza título (h1), descripción, badges humanos y comentarios simultáneamente 361ms
✓ TaskDetailPage - acciones y permisos > el líder cambia el estado reutilizando la mutation de estado 304ms
✓ TaskDetailPage - acciones y permisos > "Editar tarea" abre el formulario compartido sin salir de la ruta 385ms
✓ test/project-labels-drawer.spec.ts (18 tests) 2214ms
✓ test/task-label-chip.spec.ts (4 tests) 113ms
✓ test/hitos-section.spec.ts (12 tests) 626ms
✓ test/kanban-workspace.spec.ts (11 tests) 2140ms
✓ KanbanWorkspaceClient - Tablero/Hitos (Sección 19/29) > muestra las pestañas Tablero e Hitos, el título y el encabezado del workspace 338ms
✓ KanbanWorkspaceClient - Tablero/Hitos (Sección 19/29) > llama useProjectTasks con el idProyecto numérico y no usa proyecto.tareas 319ms
✓ test/notification-task-links.spec.ts (11 tests) 310ms
✓ test/next-routes.spec.ts (2 tests) 31ms
✓ test/api-client.spec.ts (3 tests) 12ms
✓ test/cloudinary.spec.ts (2 tests) 13ms
✓ test/form-types.spec.ts (4 tests) 10ms
✓ test/user-flows.spec.ts (2 tests) 3ms
✓ test/tasks-contract.spec.ts (11 tests) 9ms
✓ test/task-comments-service.spec.ts (6 tests) 13ms
✓ test/task-board-dnd.spec.ts (18 tests) 10ms
✓ test/labels-service.spec.ts (7 tests) 7ms
✓ test/api-error.spec.ts (9 tests) 7ms
✓ test/services.spec.ts (3 tests) 18ms
✓ test/project-roles-sheet.spec.ts (16 tests) 3256ms
✓ ProjectRolesSheet - gestión completa (Sección 23/5C) > entrada directa CREATE: abre formulario, no lista, y cancelar cierra el Sheet 479ms
✓ ProjectRolesSheet - gestión completa (Sección 23/5C) > entrada directa Edit: guardar exitosamente cierra el Sheet 318ms
✓ ProjectRolesSheet - gestión completa (Sección 23/5C) > editar desde MANAGER: cancelar y éxito regresan a LIST sin cerrar Sheet 369ms
✓ ProjectRolesSheet - gestión completa (Sección 23/5C) > crea un rol con carrera, habilidad y nivel usando los catálogos reales 731ms
✓ test/task-progress.utils.spec.ts (8 tests) 7ms
✓ test/task-board.spec.ts (25 tests) 3331ms
✓ TaskBoard > "Limpiar filtros" restablece ambos filtros 461ms
✓ test/task-form-dialog.spec.ts (23 tests) 4421ms
✓ TaskFormDialog - reutilización > el mismo componente muestra el título "Crear nueva tarea" en modo creación 369ms
✓ TaskFormDialog - reutilización > el mismo componente muestra el título "Editar tarea" en modo edición 322ms
✓ TaskFormDialog - reutilización > no existen dos formularios: los mismos campos aparecen en ambos modos 610ms
✓ TaskFormDialog - reutilización > muestra las tres secciones numeradas en ambos modos 545ms
✓ TaskFormDialog - creación > incluye asignado inicial cuando se seleccionó uno 495ms

Test Files 48 passed (48)
Tests 440 passed (440)
Start at 13:24:04
Duration 6.43s (transform 3.22s, setup 8ms, collect 20.23s, tests 30.14s, environment 23.01s, prepare 5.87s)

vernel@192:~/Documentos/IngenieriaSoftware/proyecto-ingenieria-software/apps/frontend$
```

Figura 3. Ejecución automatizada de las pruebas del frontend con Vitest: 440 pruebas aprobadas y ninguna fallida.

```
Generar cliente Prisma

Lint

1 ▶ Run npm run lint
7
8 > uvg-collab-backend@0.1.0 lint
9 > eslint "{src,test}/**/*.ts"
10
11
12 /home/runner/work/projecto-ingenieria-software/projecto-ingenieria-software/apps/backend/src/auth/auth.service.ts
13 Warning: 132:18 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
14
15 /home/runner/work/projecto-ingenieria-software/projecto-ingenieria-software/apps/backend/src/common/interceptors/audit.interceptor.ts
16 Warning: 15:71 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
17 Warning: 46:11 warning Unexpected console statement no-console
18 Warning: 52:35 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
19 Warning: 73:34 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
20 Warning: 73:40 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
21
22 /home/runner/work/projecto-ingenieria-software/projecto-ingenieria-software/apps/backend/src/evidence/evidence.controller.ts
23 Warning: 9:24 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
24
25 /home/runner/work/projecto-ingenieria-software/projecto-ingenieria-software/apps/backend/src/evidence/evidence.service.ts
26 Warning: 8:17 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
27
28 /home/runner/work/projecto-ingenieria-software/projecto-ingenieria-software/apps/backend/src/main.ts
29 Warning: 37:3 warning Unexpected console statement no-console
30
31 /home/runner/work/projecto-ingenieria-software/projecto-ingenieria-software/apps/backend/src/notifications/notifications.gateway.ts
32 Error: 46:14 error 'error' is defined but never used @typescript-eslint/no-unused-vars
33 Warning: 57:50 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
34 Warning: 61:54 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
35
36 /home/runner/work/projecto-ingenieria-software/projecto-ingenieria-software/apps/backend/src/notifications/notifications.module.ts
37 Error: 1:18 error 'forwardRef' is defined but never used. Allowed unused vars must match /^_/u @typescript-eslint/no-unused-vars
38
39 /home/runner/work/projecto-ingenieria-software/projecto-ingenieria-software/apps/backend/src/notifications/notifications.service.ts
40 Warning: 104:81 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
41 Warning: 105:46 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
42
43 /home/runner/work/projecto-ingenieria-software/projecto-ingenieria-software/apps/backend/src/projects/projects.service.ts
44 Warning: 516:48 warning Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
45 Error: 537:15 error 'totalPostulaciones' is defined but never used. Allowed unused args must match /^_/u @typescript-eslint/no-unused-vars
46
```

Figura 4: Pruebas de CI en github al momento de deployar



**All checks have passed**
4 successful checks

 CI / Backend - lint y pruebas (pull_request) Successful in 52s ...

 CI / Backend - lint y pruebas (push) Successful in 56s ...

 CI / Frontend - lint y pruebas (pull_request) Successful in 1m ...

 CI / Frontend - lint y pruebas (push) Successful in 1m ...

**No conflicts with base branch**
Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Figura 5: Pruebas de CI al momento de hacer Pull Request

Deben tenerse presentes las siguientes salvedades: no existe un script de cobertura configurado, por lo que no puede afirmarse un porcentaje de cobertura; aprobar todas las pruebas ejecutadas no significa cobertura total; faltan pruebas E2E completas con navegador y pruebas de carga; y las pruebas que requieren PostgreSQL deben ejecutarse posteriormente en un ambiente preparado.

Impacto de las pruebas en el producto

Las 1,637 pruebas aprobadas aportan evidencia de estabilidad al incremento entregado durante el Sprint 5 y reducen el riesgo de introducir regresiones al integrar nuevos cambios. Los casos seleccionados verifican reglas críticas relacionadas con autorización, asignación y cambio de estado de tareas, aislamiento entre proyectos, comentarios y notificaciones. Esto permite detectar anticipadamente errores que podrían ocasionar accesos indebidos, duplicación de avisos, pérdida de información o inconsistencias en el tablero Kanban.

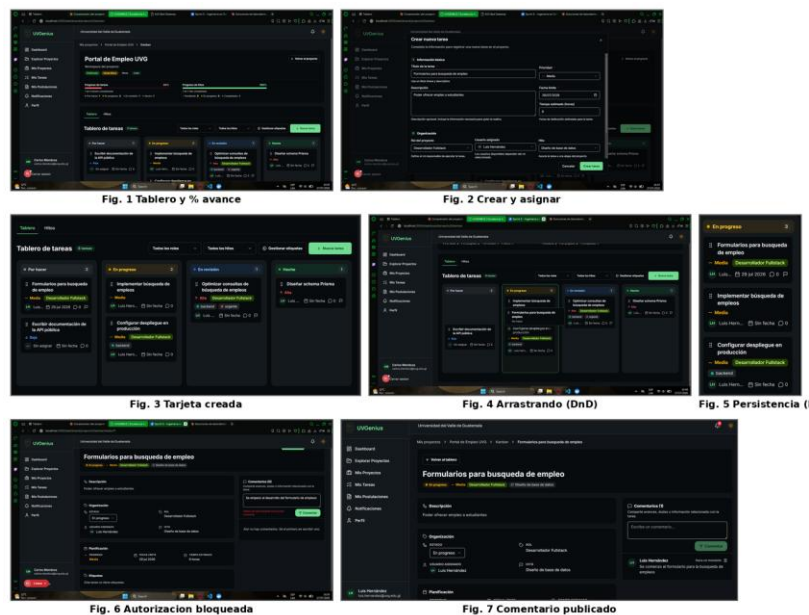


Figura 6: Validaciones manuales para las pruebas de diseño

Las diez pruebas omitidas requieren una instancia real de PostgreSQL y no representan fallas de la lógica evaluada. Sin embargo, los resultados no garantizan por sí solos la ausencia total de defectos, ya que el proyecto todavía debe complementarse con medición de cobertura, ejecución de las pruebas pendientes contra una base de datos controlada, pruebas End-to-End en navegador y pruebas de rendimiento. En conjunto, las pruebas automatizadas aumentan la confianza sobre el funcionamiento del incremento y facilitan su mantenimiento posterior.

Resultados del Sprint 5

Gráfica burndown

ia - 6 de julio de 2026 - 28 de julio de 2026

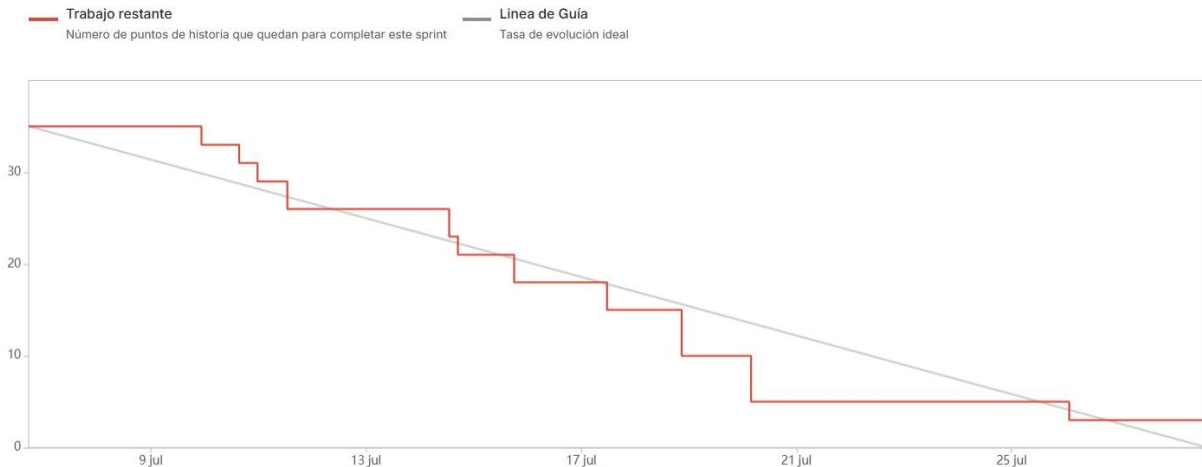


Figura 7: Grafico Burndown del Sprint 5

Interpretación de resultados

El gráfico se construyó con la fecha de resolución registrada en Jira para cada historia, por lo que refleja el cierre formal en el tablero y no una reconstrucción posterior. El equipo cerró la totalidad de los 35 puntos comprometidos dentro del periodo del sprint, alcanzando cero puntos pendientes el 28 de julio. La forma de la curva revela tres fases claramente diferenciadas.

Arranque sin cierres (6 al 8 de julio). Durante los primeros tres días la línea real permanece plana en 35 puntos mientras la ideal ya había descendido a 30. Esto se explica por la naturaleza del sprint: las primeras historias requerían construir el módulo de tareas desde cero en el backend antes de que existiera algo entregable, de modo que hubo trabajo sostenido sin ningún incremento cerrable.

Fase de alto rendimiento (9 al 20 de julio). En doce días el equipo cerró 30 de los 35 puntos, y la línea real se despegó de la ideal por debajo. El punto de máxima ventaja se alcanzó el 20 de julio, cuando quedaban 5 puntos pendientes frente a los 12.7 que preveía la línea ideal para esa fecha. En el punto medio del sprint, el 17 de julio, el equipo ya había entregado 20 de 35 puntos, es decir un 57 % del alcance. Esta fase incluye tanto las historias retomadas del Sprint 3 (HU-20, HU-22, HU-61, HU-14 y HU-59), que partían de una base de código existente, como las dos historias grandes del módulo de tareas (HU-29 y HU-30).

Meseta final (20 al 26 de julio). Durante seis días consecutivos no se cerró ningún issue y la curva permanece horizontal en 5 puntos. En ese tramo la línea ideal descendió de 12.7 a 3.2, de modo que toda la ventaja acumulada durante la fase anterior se consumió por completo. Los 5 puntos restantes correspondían a HU-67 (etiquetas y prioridades) y HT-05 (tareas transversales de entrega), que se cerraron el 26 y el 28 de julio respectivamente, esta última el mismo día de la revisión del sprint.

La lectura de conjunto es favorable en cuanto a capacidad y desfavorable en cuanto a ritmo de cierre. El equipo demostró que puede entregar 30 puntos en doce días, un ritmo muy superior al promedio histórico, pero perdió esa ventaja en la última semana. La meseta no significa que no hubiera trabajo: el historial del repositorio muestra actividad continua entre el 20 y el 23 de julio. Lo que ocurrió es que ese trabajo correspondía a historias que no admiten cierres parciales bajo la definición actual de terminado, por lo que no se reflejó en el tablero hasta completarse por entero.

En comparación con iteraciones anteriores, la distribución mejoró de forma apreciable. Los sprints 2 y 3 concentraron la mayor parte del cierre en el tramo final, mientras que aquí el grueso del alcance se entregó en la primera mitad. La palanca de mejora ya no es el ritmo de trabajo del equipo sino la granularidad de las historias: dividir las historias de cinco puntos y las tareas transversales en incrementos verificables de forma independiente evitaría mesetas de seis días y permitiría un descenso continuo hasta el cierre.

6.2 Cumplimiento de fechas planificadas por issue

Además del agregado, conviene observar la desviación individual. De los veintiocho issues del sprint, ocho se cerraron después de su fecha de vencimiento planificada y nueve se cerraron antes de tiempo. Las desviaciones más significativas fueron las siguientes.

Issue	Título	Vencimiento	Cierre real	Desviación
HU-67	Etiquetas y prioridades en las tarjetas	18/07	26/07	+8 días
T-99	Cálculo de % de avance y barra de progreso	20/07	28/07	+8 días
T-93	Endpoint de cambio de estado y orden	14/07	20/07	+6 días
T-91	Módulo tasks con CRUD de tareas	10/07	15/07	+5 días
HU-29	Crear, asignar y editar tareas	14/07	18/07	+4 días
HU-22	Notificación al líder por postulación	24/07	10/07	-14 días
HU-32	Barra de avance del proyecto	20/07	11/07	-9 días
HU-59	Proyectos destacados en el dashboard	24/07	17/07	-7 días

El patrón es consistente: las historias con mayor componente de construcción nueva se retrasaron y las que partían de código preexistente se adelantaron. Esto confirma que la estimación del equipo es precisa para trabajo incremental pero optimista para módulos construidos desde cero, y sugiere aplicar un factor de holgura a las historias de cinco puntos en la planificación del Sprint 6.

Métrica de velocidad

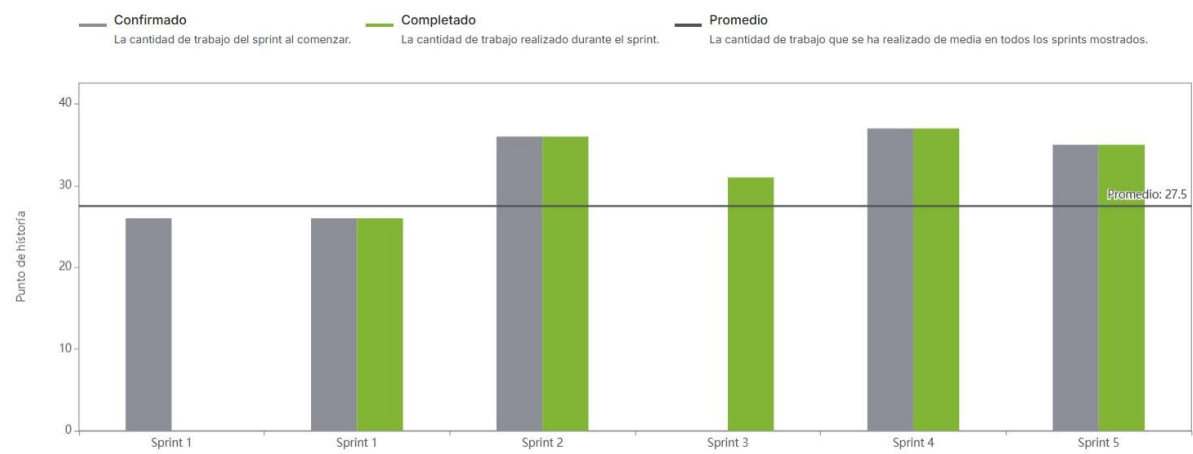


Figura 8: Grafico de velocidad del Sprint 5

Interpretación de resultados

Sprint	Periodo	SP comprometidos	SP completados	Cumplimiento
Sprint 1	30/03 – 05/04	26	26	100 %
Sprint 2	13/04 – 22/04	35	35	100 %
Sprint 3	23/04 – 11/05	34	34	100 %
Sprint 4	13/05 – 27/05	31	31	100 %
Sprint 5	06/07 – 28/07	35	35	100 %
Total promedio	/ —	161 (32.2)	161 (32.2)	100 %

Discusión de la velocidad

La velocidad del Sprint 5 fue de 35 puntos de historia, cifra que iguala el máximo histórico alcanzado en el Sprint 2 y que se ubica un 8.7 % por encima del promedio de 32.2 puntos. Tras el descenso del Sprint 4 a 31 puntos, la iteración actual recupera el nivel superior del rango.

Vista en conjunto, la serie muestra una velocidad estabilizada. Después del Sprint 1, que funcionó como línea base mientras el equipo se familiarizaba con el stack tecnológico, los cuatro sprints siguientes se mueven en una banda estrecha entre 31 y 35 puntos, con una desviación de apenas dos puntos alrededor de la media. Esta estabilidad es más valiosa que una velocidad creciente, porque significa que la capacidad del equipo es predecible y que las estimaciones de planificación pueden apoyarse en ella con confianza.

Debe señalarse una precisión importante sobre la lectura de esta métrica. Cinco de las historias del Sprint 5, específicamente HU-14, HU-20, HU-22, HU-59 y HU-61, que suman 12 puntos, corresponden a trabajo que se había contabilizado previamente en la velocidad del Sprint 3. En términos de valor nuevo entregado al producto, el Sprint 5 aportó 23 puntos, y los 12 restantes corresponden a la corrección de historias que se habían cerrado prematuramente. Reconocer esto explícitamente es más útil que reportar 35 puntos sin matices, porque señala dónde está el problema real: no en la capacidad de producción del equipo, sino en el criterio con el que se declara terminada una historia.

Para el Semestre 2 se recomienda planificar sobre un rango de 30 a 33 puntos por sprint en lugar de sobre el máximo histórico, y medir la velocidad únicamente sobre historias que hayan superado la validación en el entorno de producción.

Resultados del cumplimiento del Sprint Backlog

Tareas concluidas

Las doce historias comprometidas fueron completadas e integradas a la rama main del repositorio, junto con sus diecisiete subtareas asociadas.

ID	Historia	SP	Responsable	Cierre en Jira
HT-05	Tareas transversales Sprint 5	3	Angel Sanabria	28/07
HU-29	Crear, asignar y editar tareas del proyecto	5	Vernel Hernández	18/07
HU-30	Tablero Kanban del proyecto con drag & drop	5	Samuel Robledo	20/07
HU-67	Etiquetas y prioridades visibles en las tarjetas	2	Vernel Hernández	26/07

HU-09	Vista «Mis tareas» transversal a los proyectos	3	Saul Castillo	14/07
HU-31	Comentarios en tareas del proyecto	2	Derek Coronado	10/07
HU-32	Barra de avance del proyecto según tareas	3	Derek Coronado	11/07
HU-14	Recuperación de contraseña vía administrador	3	Angel Sanabria	15/07
HU-20	Cancelar postulación a un proyecto	2	Samuel Robledo	09/07
HU-22	Notificación al líder por nueva postulación	2	Derek Coronado	10/07
HU-61	Autocompletar correo institucional en el registro	2	Saul Castillo	14/07
HU-59	Proyectos destacados en el dashboard	3	Angel Sanabria	17/07

Total, completado: 35 de 35 puntos de historia comprometidos, equivalente al 100 % del alcance.

Tareas en proceso al cierre del sprint

Ninguna historia del Sprint Backlog quedó en estado En progreso al cierre del sprint.

Tareas planificadas que no se pudieron concluir

No se registraron historias planificadas sin concluir durante el Sprint 5. El alcance comprometido en la reunión de planificación se entregó en su totalidad y no se añadieron historias al sprint una vez iniciado.

Indicador numérico del éxito del sprint

Se aplica el Índice de Éxito del Sprint (IES) definido por el equipo en el Sprint 2, manteniendo idénticos los pesos de cada dimensión para permitir la comparación entre iteraciones:

$$\text{IES} = (C \times 0.30) + (V \times 0.15) + (Q \times 0.15) + (D \times 0.10) + (P \times 0.15) + (U \times 0.15)$$

Dimensión	Peso	Calif.	Ponderado	Justificación
Cumplimiento del alcance (C)	30 %	10	3.00	35 de 35 SP entregados, equivalente al 100 %. Ninguna historia quedó sin concluir.
Tendencia de velocidad (V)	15 %	9	1.35	Recuperación de 31 a 35 SP. Se descuenta un punto porque 12 de los 35 SP corresponden a re-trabajo del Sprint 3.
Calidad del incremento (Q)	15 %	9	1.35	1,637 pruebas aprobadas y ninguna fallida. Las 10 omitidas requieren base de datos real y no representan defectos de lógica.
Distribución del trabajo (D)	10 %	5	0.50	Arranque de cuatro días sin cierres, dos mesetas intermedias y una concentración severa de contribuciones en dos integrantes.
Prácticas de ingeniería (P)	15 %	8	1.20	99 archivos de prueba automatizada y disciplina de ramas por issue de Jira. No hay medición de cobertura ni pruebas E2E de navegador.
Retroalimentación de usuarios (U)	15 %	8	1.20	Validación manual del módulo de tareas y demostración al Product Owner. Falta ampliar la muestra de usuarios finales.
TOTAL	100 %	—	8.60	Sprint exitoso con deuda identificada en la distribución del trabajo.

Conclusiones

El Sprint 5 se clasifica como exitoso. La calificación se sostiene sobre el cumplimiento íntegro del alcance y sobre el salto cualitativo en pruebas automatizadas, que era la principal medida correctiva heredada de las retrospectivas del Semestre I. El incremento entregado, con el módulo completo de gestión de tareas y el tablero Kanban, es el más sustantivo del proyecto hasta la fecha y acerca el producto a su propuesta de valor original: dejar de ser un catálogo de proyectos para convertirse en una herramienta de seguimiento de trabajo.

La calificación no alcanza un valor superior por dos razones concretas y accionables. La primera es la distribución del trabajo, que es la dimensión de peor desempeño con 5 sobre 10 y que se manifiesta tanto en la forma del burndown como en la concentración de contribuciones al repositorio en dos de los cinco integrantes. La segunda es que, pese a la cantidad de pruebas construidas, el proyecto aún no cuenta con medición de cobertura ni con pruebas de extremo a extremo ejecutadas en navegador, por lo que la confianza sobre el incremento tiene un techo verificable.

Retrospectiva del Sprint 5

Aspectos positivos

- Incremento funcional más ambicioso del proyecto. El módulo de gestión de tareas entregado en este sprint, que incluye creación y asignación de tareas, tablero Kanban con arrastrar y soltar, etiquetas y prioridades, comentarios, cálculo de avance y vista transversal de tareas propias, convierte a la plataforma en una herramienta de seguimiento de trabajo y no solamente en un catálogo de proyectos.
- Salto real en pruebas automatizadas. Se pasó de una cobertura testimonial a 97 archivos de especificación y 1,637 pruebas aprobadas entre backend y frontend, cubriendo reglas de autorización, condiciones de carrera en la asignación de tareas e integración de arrastrar y soltar del tablero. Esta era la medida correctiva pendiente desde la retrospectiva del Sprint 2 y finalmente se atendió.
- Resolución de un bloqueo técnico heredado. La historia de recuperación de contraseña llevaba dos sprints sin funcionar en producción por una restricción externa al equipo. En lugar de insistir sobre la misma solución, se rediseñó el flujo sobre infraestructura propia, lo cual demuestra capacidad de replanteo ante restricciones que no se pueden negociar.
- Quinto sprint consecutivo con cumplimiento del 100 % del alcance comprometido. No se añadieron historias al sprint una vez iniciado, lo cual evidencia que la reunión de planificación fue efectiva y que la estimación del equipo es confiable.
- Cero deuda de integración al cierre. La rama develop quedó completamente fusionada en main, sin commits divergentes, y el sistema en producción responde correctamente tanto en la ruta del frontend como en la del backend.

Aspectos negativos

- Distribución del trabajo severamente desequilibrada. El análisis del historial de contribuciones muestra que dos integrantes concentraron cerca del 80 % de los commits del sprint, mientras que tres quedaron por debajo del umbral esperado, y dos de ellos no registraron actividad en el repositorio durante las últimas dos semanas del sprint. Esta es la falla más grave de la iteración y afecta directamente tanto la evaluación individual como la sostenibilidad del ritmo del equipo.
- Ocho de los veintiocho issues del sprint se cerraron después de su fecha de vencimiento planificada, con desviaciones de entre uno y ocho días. Las mayores corresponden a HU-67 y a la subtarea T-99, ambas cerradas ocho días tarde. Esto indica que la estimación por subtarea fue optimista para el trabajo de mayor complejidad técnica.
- Contribuciones concentradas en ventanas cortas. Incluso entre quienes cumplieron, el trabajo se agrupó en bloques de dos o tres días consecutivos en lugar de distribuirse a lo largo del sprint, lo que reproduce el patrón señalado en retrospectivas anteriores y comprime el tiempo disponible para pruebas de integración.
- Historias declaradas terminadas sin validación en producción. Cinco historias cerradas en el Sprint 3 tuvieron que retomarse en este sprint porque no funcionaban de extremo a extremo en el entorno productivo. Esto infló artificialmente la velocidad reportada del Sprint 3 y consumió capacidad del Sprint 5 que estaba destinada a funcionalidad nueva.
- La subtarea T-89 se marcó como Finalizada en Jira el 8 de julio, pero el archivo de despliegue nunca fue modificado: la variable de entorno sigue sin valor en el repositorio. Un elemento del tablero pasó a Done sin que su criterio de aceptación se cumpliera, y el error permaneció invisible durante veinte días porque el sistema seguía funcionando por una ruta alterna.
- Ausencia de medición de cobertura y de pruebas de extremo a extremo en navegador. La suite construida es amplia, pero sin un script de cobertura configurado no es posible afirmar qué porción del código está efectivamente verificada, y las diez pruebas que requieren base de datos permanecen sin ejecutarse por falta de un entorno preparado.
- La documentación del repositorio quedó desactualizada respecto al estado real del proyecto: el archivo README continúa identificando el curso y el semestre anteriores y reporta el Sprint 4 como la última iteración cerrada, y no existe una carpeta versionada para los artefactos del Sprint 5.

Medidas correctivas para el Sprint 6

1. Establecer un compromiso mínimo de contribución semanal por integrante, verificable en el historial del repositorio, y revisarlo en la reunión diaria de cada miércoles. Si un integrante no ha registrado actividad al llegar ese punto de control, el equipo redistribuye o desbloquea su tarea antes de que el retraso se acumule.
2. Ampliar la Definición de Terminado: una historia solo pasa al estado Done cuando ha sido verificada en el entorno de producción, no únicamente en local. Esta medida ataca directamente la causa del re-trabajo de las cinco historias del Sprint 3.
3. Configurar un script de cobertura con Vitest y establecer un umbral mínimo, de modo que el porcentaje de código verificado sea una cifra reportable y no una afirmación cualitativa.
4. Levantar un contenedor efímero de PostgreSQL dentro del pipeline de integración continua para ejecutar las diez pruebas que hoy quedan omitidas por depender de una base de datos real.
5. Dividir las historias de cinco puntos en subtarefas con entregable verificable de forma independiente, de modo que el burndown pueda descender de manera continua en lugar de por saltos al completar historias completas.
6. Realizar una auditoría del código antes de la reunión de planificación para confirmar qué historias del backlog ya están implementadas y cuáles requieren trabajo real, evitando planificar sobre supuestos incorrectos.
7. Rotar entre los integrantes el rol de responsable de la integración y del cierre del sprint, de modo que la carga de consolidación y despliegue no recaiga siempre sobre la misma persona.
8. Actualizar el archivo README y crear la carpeta versionada del Sprint 5 con el informe, los formularios de gestión de tiempo y el enlace al documento colaborativo, siguiendo la estructura de los sprints anteriores.

Interacción con el usuario y Product Owner

El rol de Product Owner del proyecto es desempeñado por Norman Aguirre, quien es el presidente de la asociación de estudiantes de Ciencias de la Computacion y Tecnologias de la Informacion.

Durante el Sprint 5 se realizó una demostración del incremento el día 28 de julio de 2026, en la que se presentó el flujo completo del módulo de gestión de tareas: creación y asignación de una tarea dentro de un proyecto, movimiento de la tarjeta entre columnas del tablero Kanban, publicación de un comentario y visualización del porcentaje de avance del proyecto.

Comentarios recibidos

- Me gusto la UI, me gustaria que hubiese un portal para las asociaciones y que ellos puedan publicar proyectos
- Me parece interesante lo del tablero kanban

